

# **Лабораторная работа №8**

**Целочисленная арифметика многократной точности**

Тазаева Анастасия Анатольевна

# Содержание

|          |                                                                |           |
|----------|----------------------------------------------------------------|-----------|
| <b>1</b> | <b>Цель работы</b>                                             | <b>5</b>  |
| <b>2</b> | <b>Задание</b>                                                 | <b>6</b>  |
| <b>3</b> | <b>Выполнение лабораторной работы</b>                          | <b>7</b>  |
| 3.1      | Алгоритм 1 (сложение неотрицательных целых чисел) . . . . .    | 7         |
| 3.2      | Алгоритм 2 (вычитание неотрицательных целых чисел) . . . . .   | 8         |
| 3.3      | Алгоритм 3 (умножение неотрицательных целых чисел столбиком) . | 10        |
| 3.4      | Алгоритм 4 (быстрый столбик) . . . . .                         | 12        |
| <b>4</b> | <b>Выводы</b>                                                  | <b>14</b> |
|          | <b>Список литературы</b>                                       | <b>15</b> |

## Список иллюстраций

|     |                                                                                   |    |
|-----|-----------------------------------------------------------------------------------|----|
| 3.1 | Алгоритм 1 (сложение неотрицательных целых чисел). Пример . . .                   | 8  |
| 3.2 | Алгоритм 2 (вычитание неотрицательных целых чисел). Пример . . .                  | 10 |
| 3.3 | Алгоритм 3 (умножение неотрицательных целых чисел столбиком).<br>Пример . . . . . | 12 |
| 3.4 | Алгоритм 4 (быстрый столбик). Пример . . . . .                                    | 13 |

## **Список таблиц**

# 1 Цель работы

Ознакомиться с алгоритмами для выполнения арифметических операций.

## 2 Задание

1. Реализовать на языке программирования Julia алгоритмы для выполнения арифметических операций.

## 3 Выполнение лабораторной работы

### 3.1 Алгоритм 1 (сложение неотрицательных целых чисел)

Написан программный код на языке Julia [1], реализующий сложение неотрицательных целых чисел:

```
function algoritm1(u::Vector{Int}, v::Vector{Int}, b::Int)
    n = length(u)
    w = zeros(Int, n + 1) # + w0
    # step 1
    k = 0 # perenos
    j = n # razryad
    # step 2 & 3
    #println("j", "\t", "w[j]", "\t", "u[j]", "\t", "v[j]", "\t", "k")
    #println(j, "\t", w[j], "\t", u[j], "\t", v[j], "\t", k)
    while j > 0
        # step 2
        w[j+1] = mod(u[j] + v[j] + k, b)
        k = div(u[j] + v[j] + k, b)
        #println(j, "\t", w[j], "\t", u[j], "\t", v[j], "\t", k)
        # step 3
```

```

        j -= 1
    end
    w[1] = k
    return w
end

```

Получен следующий результат выполнения программного кода (рис. 3.1).

```

u = [1,2,3]
v = [4,5,6]
b = 10
result = algoritm1(u,v,b)
println("Сумма равна ", result)

```

| j | w[j] | u[j] | v[j] | k |
|---|------|------|------|---|
| 3 | 0    | 3    | 6    | 0 |
| 3 | 0    | 3    | 6    | 0 |
| 2 | 0    | 2    | 5    | 0 |
| 1 | 0    | 1    | 4    | 0 |

Сумма равна [0, 5, 7, 9]

Рисунок 3.1: Алгоритм 1 (сложение неотрицательных целых чисел). Пример

## 3.2 Алгоритм 2 (вычитание неотрицательных целых чисел)

Написан программный код на языке Julia [1], реализующий вычитание неотрицательных целых чисел:



```

function algoritm2(u::Vector{Int}, v::Vector{Int}, b::Int)
    n = length(u)
    w = zeros(Int, n)
    # step 1
    k = 0 # zaem
    j = n # razryad
    # step 2 & 3
    println("j", "\t", "w[j]", "\t", "u[j]", "\t", "v[j]", "\t", "k")
    println(j, "\t", w[j], "\t", u[j], "\t", v[j], "\t", k)
    while j > 0
        # step 2
        w[j] = mod(u[j] - v[j] + k, b)
        k = div(u[j] - v[j] + k, b)
        println(j, "\t", w[j], "\t", u[j], "\t", v[j], "\t", k)
        # step 3
        j -= 1
    end
    return w
end

```

Получен следующий результат выполнения программного кода (рис. 3.2).

```

b = [1,2,3]
a = [4,5,6]
c = 10
result2 = algoritm2(a,b,c)
println("Разность равна ", result2)

```

| j | w[j] | u[j] | v[j] | k |
|---|------|------|------|---|
| 6 | 3    | 0    |      |   |
| 3 | 3    | 6    | 3    | 0 |
| 2 | 3    | 5    | 2    | 0 |
| 1 | 3    | 4    | 1    | 0 |

Разность равна [3, 3, 3]

Рисунок 3.2: Алгоритм 2 (вычитание неотрицательных целых чисел). Пример

### 3.3 Алгоритм 3 (умножение неотрицательных целых чисел столбиком)

Написан программный код на языке Julia [1], реализующий умножение неотрицательных целых чисел столбиком:

```

function algoritm3(u::Vector{Int}, v::Vector{Int}, b::Int)
    n = length(u)
    m = length(v)
    w = zeros{Int, n + m}

    # step 1
    fill!(w[m+1:end], 0)
    j = m
    # step 2-6

```

```

while j > 0
    # step 2: if vj = 0 -> propyskaem razryad
    vj = v[j]
    if vj == 0
        j -= 1
        continue
    end
    # step 3
    i = n # index razryada 4isla u
    k = 0 # perenos
    # step 4 - 5
    while i > 0
        t = u[i] * vj + w[i + j] + k
        w[i + j] = mod(t, b) #res
        k = div(t, b) # new perenos
        i -= 1 # sled.razryad
    end
    if k > 0
        w[j] = k
    end
    j -= 1
end
return w
end

```

Получен следующий результат выполнения программного кода (рис. 3.3).

[50]:

```
u = [1, 2]
v = [3, 4]
b = 10

result = algoritm3(u, v, b)
println("Результат: ", result)
```

Результат: [0, 4, 0, 8]

Рисунок 3.3: Алгоритм 3 (умножение неотрицательных целых чисел столбиком).  
Пример

### 3.4 Алгоритм 4 (быстрый столбик)

Написан программный код на языке Julia [1], реализующий умножение быстрым столбиком:

```
function algoritm4(u::Vector{Int}, v::Vector{Int}, b::Int)
    n = length(u)
    m = length(v)
    w = zeros{Int}(m + n)
    # step 1
    t = 0
    # step 2
    for s in 0:(m + n - 1)
        # step 3
        for i in 0:s
            if n - i > 0 && m - s + i > 0
                t += u[n - i] * v[m - s + i]
            end
        end
    end
end
```

```

        end
    end
    # step 4
    w[m + n - s] = mod(t, b)
    t = div(t, b)
end
return w
end

```

Получен следующий результат выполнения программного кода (рис. 3.4).

```

u = [1, 2]
v = [3, 4]
b = 10

result = algorithm4(u, v, b)
println("Результат: ", result)

```

Результат: [0, 4, 0, 8]

Рисунок 3.4: Алгоритм 4 (быстрый столбик). Пример

## 4 Выводы

Ознакомилась с алгоритмами для выполнения арифметических операций.

# Список литературы

- [1] *Julia 1.10 Documentation*. Англ. 2024. URL: <https://docs.julialang.org/en/v1/>.